

Concurrency & Transactions

Mulakat Hazirlik Notlari

ACID · Transaction · Thread Safety · Locks
Race Condition · Deadlock · JMM · Atomics

Bir DB'de 100 transaction birbirine girerken, JVM'de 1000 thread ayni veriyi doverken, neden sistemler patlamiyor? Ya patladiklarinda ne oluyor? Bu doküman onu anlatiyor.

ARGUS / HR-WEB orneklerinin esliginde

1. Transaction & ACID

"Ya hepsi olacak, ya hicbiri. Yari kalmis hicbir sey gormeyecegiz."

Once sezgi

ATM'ye gittin, A hesabından B hesabına 1000 TL gonderiyorsun. Bilgisayarın yaptigi sey iki adim:

(1) A hesabından 1000 TL dus.

(2) B hesabına 1000 TL ekle.

Adim 1 yapildi, sonra elektrik gitti. Adim 2 yapilmadi. A'nin 1000 TL'si gitti, B'ye gelmedi, kayboldu. **Bu felaketin yasanmaması için transaction icat edildi.**

Transaction soyle der: "Bu iki adimi bir paket olarak yapacagim. Ikisi de basariyla bitmedikce hicbiri kalici olmayacak. Eger ortada bir sey patlarsa, ilk adimi geri alacagim."

ARGUS ornegi: izin onayi tek bir is gibi gozukur ama altinda 5 islem var:

Izin kaydini "onaylandi" yap

Izin gecmis tablosuna kayit at

Personelin kullanilabilir izin gunu sayisini azalt

Audit log yaz

Bildirim kuyruguna mail ekle

Eger 3. adim DB constraint'ine takilirsä, ilk 2 adim da geri alinmalı. Yoksa izin onaylanmış gozukur ama personelin gun sayisi azalmamis olur — tutarsizlik. Transaction tum 5'ini **tek paket** haline getirir.

ACID — dort soz

ACID, transaction'ın vermesi gereken 4 garantinin baş harfleridir. Mulakatci "ACID nedir?" diye sordugunda her birini ornekle anlat:

A — Atomicity (atomiklik)

"Ya hep, ya hic." Transaction içindeki tum islemler ya butunuyle uygulanir ya da hicbiri uygulanmaz. Ortada yari uygulanmis bir durum DB'ye yazilmaz.

Yukaridaki izin onayi orneginde 3. adim hata verirse, DB **otomatik olarak** ilk 2 adimi geri alır (rollback). Sanki transaction hic baslamamis gibi olur.

```
@Transactional
public void izinOnayla(Long izinId) {
    izinRepo.updateStatus(izinId, "ONAYLANDI"); // adim 1
    izinGecmisRepo.kaydet(...); // adim 2
    personelRepo.izinGunuAzalt(personelId, gunSayisi); // adim 3
    auditLog.kaydet(...); // adim 4
    bildirimKuyrugunu.ekle(...); // adim 5
}
```

```
} // Herhangi biri exception fırlatırsa, hepsi rollback olur
```

C — Consistency (tutarlılık)

"Kuralları bozma." Transaction'dan önce veri kurallara uygunsa, transaction'dan sonra da uygun olmalı. Burada "kurallar" demek: foreign key constraint'lari, check constraint'lari, unique index'ler, business rule'lar.

Örnek: TBECDT tablosunda FK var, var olmayan bir CBECSVY'ye referans verme yasak. Transaction içinde böyle bir insert yapmaya kalkarsan, DB transaction'ı fail ettirir, rollback olur. **Yarı tutarlı** bir state yaratmana izin verilmez.

Atomicity ile karıştırma: Atomicity "hepsi ya da hiçbirini" diyor (teknik), Consistency "sonuç kurallara uymalı" diyor (mantıksal). Atomicity araç, Consistency hedef.

I — Isolation (yalıtım)

"Es zamanlı transaction'lar birbirini rahatsız etmesin." Sen izin onaylarken, başka bir HR uzmanı aynı personelin izin bilgisini ediyor. Sen yarı işlem yaparken o senin yarı halini görmemeli; ya hiç görmeli ya da bittikten sonrasını görmeli.

Bu en kritik, en karmaşık özelliktir. Isolation seviyelerine birazdan tek tek ineceğiz.

D — Durability (kalıcılık)

"Commit edildikten sonra kayboldu olmasın." Transaction commit olduktan sonra, sistem çökse, elektrik gitse, disk arızalansa bile o veri kaybolmamalı. DB normalde commit öncesi **write-ahead log**'a (WAL veya redo log) yazar — bu sayede crash sonrası kurtarma mümkün.

Pratikte ne demek: `commit()` dönüş aldığın anda, kayıtlı demektir. Sistem her ne olursa olsun bu veri orada olacak (disk değil, replica veya yedek yoksa hard disk patlarsa başka — o ayrı konu).

Hafıza taktiği: ACID kelimesinin kendisi anlamlı olarak geç — A: hep/hic, C: kural, I: yalıtım, D: kalıcı. Mulakatçı "ACID derken hangisi en zor?" diye sorabilir. **Cevap: Isolation** — çünkü diğerleri DB engine'i için teknik ama isolation senin seçimini gerektirir (level seçimi).

Isolation level'lar — burası en sık sorulan kısım

Isolation "tam yalıtım" pahalı bir özelliktir. Birbirine paralel transaction'ları tamamen izole etmek demek, onları neredeyse seri çalıştırmak demek — performans yerlerde sürünür. O yüzden SQL standardi 4 seviye tanımladı:

1. READ UNCOMMITTED — "her şeyi görürüm"

En zayıf. Bir transaction başka bir transaction'ın **commit etmediği** değişikliklerini bile görebilir. Bu **Dirty Read** denir. Pratikte neredeyse hiçbir uygulama kullanmaz — tehlikeli.

2. READ COMMITTED — "sadece commit edilmişleri görürüm"

Sadece commit edilmiş veriyi görür. Dirty read yok. Ama bir başka sorun var: **Non-Repeatable Read**. Aynı satırı iki kez okursan, arada baskası commit etmiş olabilir, ikinci okumada farklı sonuç gelir.

SQL Server, Oracle, PostgreSQL default'u budur. Pratik denge: dirty read olmasın ama performans çok da kötüye gitmesin.

3. REPEATABLE READ — "okudugum okudugum gibi kalır"

Bir transaction içinde aynı satırı kaç kez okursan oku, aynı sonucu alırsın. Non-repeatable read yok. Ama yine yeni bir sorun: **Phantom Read**. Yeni satırlar (insert) eklenmiş olabilir. Range query'ler (WHERE tarih BETWEEN ...) tekrar çalıştırıldığında farklı sayıda satır döner.

MySQL InnoDB default'u budur.

4. SERIALIZABLE — "tek başıma sanki"

En kati. Transaction'lar sanki seri çalışmış gibi davranır. Hiçbir anomali yok. **Ama performans en kötü.** Çoğu RDBMS bunu kati lock'lar veya serialization detection ile sağlar.

Anomali fenomenlerini örneklerle görelim

Dirty Read

T1, izin günü sayısını 5'ten 0'a değiştirdi ama daha commit etmedi. T2 aynı veriyi okur, 0 görür — "izin yok kardesim" der. T1 sonra rollback eder, gerçek değer 5'miş. **T2 kullanıcıya yanlış bilgi verdi.**

Non-Repeatable Read

T1, personel.maaş okur, 5000 görür. Bu sırada T2 maaşı 6000 yapar ve commit eder. T1 daha sonra aynı satırı tekrar okur, 6000 görür. **Aynı transaction içinde tutarsiz değerler.**

Phantom Read

T1, "Tosyali şirketine 2025'te alınan izinleri say" der, 10 sonuç gelir. T2 yeni bir izin ekler (Tosyali, 2025) ve commit eder. T1 aynı query'yi tekrar çalıştırır, 11 sonuç gelir. **Yeni satır "hayalet" gibi belirir.**

Mulakat tablosu — ezbere bil:

READ UNCOMMITTED: Dirty + Non-Rep + Phantom → hepsi olur

READ COMMITTED: Non-Rep + Phantom → dirty çözülür

REPEATABLE READ: Sadece Phantom → non-rep da çözülür

SERIALIZABLE: Hiçbiri yok → her şey çözülür

Spring @Transactional — pratikteki kullanim

```
@Service
public class IzinService {

    @Transactional
    public void izinOnayla(Long izinId) {
        // Bu metod icindeki tum DB islemleri tek transaction
        // Exception firlarsa otomatik rollback
        // Normal donerse otomatik commit
    }
}
```

Propagation (yayilim)

Bir @Transactional metod, zaten bir transaction icindeyken cagrildiginda ne yapacak? Propagation bunu belirler.

REQUIRED (default): Var olan transaction varsa katil, yoksa yeni bir tane basla. %95 vakanin cozumu budur.

REQUIRES_NEW: Mevcut transaction'i askiya al, kesinlikle yeni bir tane basla. Ic islem dis islemden **bagimsiz** commit/rollback olur. Audit log gibi - dis islem rollback olsa bile log kalmasi gerekiyorsa.

NESTED: Savepoint kullanir. Ic islem rollback olursa sadece o geri alinir, dis devam eder.

SUPPORTS: Varsa katil, yoksa transaction'siz devam et.

MANDATORY: Var olmalı, yoksa exception.

NEVER: Var olmamalı, varsa exception.

NOT_SUPPORTED: Varsa askiya al, transaction'siz calis.

Spring @Transactional'in 3 olumcul tuzagi

TUZAK 1: Sadece public metodlarda calisir. Cunku Spring proxy uzereinden cagiriyor (design patterns PDF'inde Proxy bolumunu hatirla). CGLIB private metodu override edemez. Private veya protected'a @Transactional koyarsan, **sessizce gormezden gelinir** — hata bile vermez, kotusu bu.

TUZAK 2: Self-invocation calismaz. Ayni sinif icinde this.metod() ile cagrilan @Transactional metod, proxy'yi bypass eder. Transaction baslamaz.

```
@Service
public class IzinService {

    public void allaha(Long id) {
        this.izinOnayla(id); // proxy BYPASS! @Transactional calismaz
    }

    @Transactional
    public void izinOnayla(Long id) {
        // ...
    }
}
```

Cozum: Self-injection, AopContext.currentProxy() veya metodu baska servise tasi.

TUZAK 3: Default rollback sadece RuntimeException'da olur! Checked exception (Exception, IOException, SQLException) firlattiginda otomatik rollback OLMAZ. Transaction commit edilir.

```
// Bu kod cilgin: checked exception firladiginda commit olur
@Transactional
public void izinOnayla(Long id) throws BusinessException {
    izinRepo.update(...);
    throw new BusinessException("hata"); // ROLLBACK YAPMAZ!
}

// Cozum:
@Transactional(rollbackFor = Exception.class)
public void izinOnayla(Long id) throws BusinessException {
    // simdi her exception'da rollback yapar
}
```

Mulakat sorulari — Transaction & ACID

"Isolation level'lerin pratikte hangisini secersin?"

READ COMMITTED (cogu RDBMS default'u). Pratikte dirty read'i onler, performansi bozmaz. Eger ayni transaction icinde tutarli okuma garantili lazimsa REPEATABLE READ'e cik. SERIALIZABLE'i ancak finansal kritik bir senaryoda secerim — overhead yuksek.

"Atomicity ile Consistency farki ne?"

Atomicity teknik bir garantidir: "hepsi ya da hicbiri". Consistency mantiksal bir hedeftir: "sonuc kurallara uymali". Atomicity Consistency'nin **araclarindan** biridir — transaction yari kalmasin diye atomik yapariz, boylece tutarsiz state olusmaz.

"@Transactional ile select yapmak mantikli mi?"

Evet. @Transactional(readOnly = true) ile yaparsin. ReadOnly Hibernate'e "dirty checking yapma, flush yapma" der — performans kazanirsin. Ayrica REPEATABLE READ gerekiyorsa ("raporu hazirlarken veri donmesin") transaction olmadan saglanmaz.

"Phantom read'i REPEATABLE READ'de cozemiyor muyduk?"

Standart REPEATABLE READ phantom'i cozmez. **Ama MySQL InnoDB** next-key locking ile pratikte phantom'i da engeller — yani MySQL'in REPEATABLE READ'i standarttan daha saglam. PostgreSQL'in REPEATABLE READ'i ise snapshot isolation kullanir, phantom genelde gozlemlenmez ama write skew anomaly cikabilir.

"Bir transaction icinde 10000 row update edersem ne olur?"

DB tum bu degisiklikleri undo/redo log'da tutar, transaction sonuna kadar lock'lari elinde tutar. Memory ve lock baskisi yaratir, diger transaction'lari yavaslatilir. Cozum: **batch'leme** — 1000'lik gruplar halinde commit et. Ama batch'leme atomicity'yi bozar (kismi basari mumkun olur), bunu bilerek yapmali.

"Long-running transaction neden kotudur?"

Lock'lari uzun sure tutar, diger transaction'lar bekler. Connection pool tuketir. Undo log buyur. Crash recovery uzar. Replica lag artar. Prensip: **transaction'i mumkun oldugu kadar kisa tut**, hesaplamayi disarida yap, DB'ye sadece son halini yansit.

2. Thread & Race Condition

İki adam aynı anda aynı veriye dokunursa ne olur? Patlama.

Once sezgi

Thread, programın yürüten bir koldur. Tek thread'li program tek elli adam gibi — aynı anda bir iş yapar. Multi-thread program 10 elli adam — paralel iş yapar.

Niye lazım? Modern CPU'larda 8+ çekirdek var, kullanmazsan israf. Web server her request için ayrı bir thread atar, yoksa ikinci request birincisi bitmeden basılamaz. Tomcat/Jetty'nin thread pool'u bu yüzden var.

ARGUS örneği: Sen onay butonuna bastığında bu bir thread. Aynı anda başka bir HR uzmanı başka bir izin onaylar, o ayrı thread. Ucuncusu rapor çıkarıyor, dördüncüsü bulk upload yapıyor. Hepsi paralel — harika ama tehlikeli.

Tehlike ne? Hepsi **aynı veriye** dokunmaya baslarsa — TSIRPRM cache, TBECDET tablosu, IzinService instance'i — işler karışır.

Race Condition — asil gunah

İki thread **aynı veriye** dokunup, birinin sonucu diğerinin yapısına bağlı ise: **race condition**. Yarismaca: kim daha hızlıysa, sonuç onun istediği gibi olur. Diğer thread'in çalışması **kaybolur**.

Klasik örnek — counter

```
class Counter {  
    private int count = 0;  
  
    public void increment() {  
        count++;    // <-- bomba burada  
    }  
}
```

count++ masum gözüküyor değil mi? Aslında **3 işlem**:

1. count'u memory'den oku (READ)
2. 1 ekle (MODIFY)
3. geri yaz (WRITE)

İki thread aynı anda increment çağırırsa:

T1: count'u oku -> 5	T2: count'u oku -> 5
T1: 1 ekle -> 6	T2: 1 ekle -> 6
T1: yaz -> 6	T2: yaz -> 6

İki increment yaptık, sonuç **6** oldu. Olması gereken **7**ydi. **Bir update kayboldu**. İşte race condition bu — birinin yaptığı iş diğerinin üzerine yazıldı ve buharlaştı.

ARGUS senaryosu

İki HR uzmanı aynı personelin izin gün sayısını güncelliyor:

Personelin TBLPER.IZIN_GUNU değeri 15.

HR1: "5 gün izin onayladım, 15'i 10 yap"

HR2: aynı anda "3 gün izin onayladım, 15'i 12 yap"

```
HR1 thread: SELECT izin_gunu FROM tblper WHERE id=42 -> 15
HR2 thread: SELECT izin_gunu FROM tblper WHERE id=42 -> 15 (HR1 daha yazmadı)
HR1 thread: UPDATE tblper SET izin_gunu = 15-5 WHERE id=42 -> 10
HR2 thread: UPDATE tblper SET izin_gunu = 15-3 WHERE id=42 -> 12
```

Sonuç: 12. Olması gereken: 7 (15-5-3). 5 gün izin buharlaştı.

Buna **Lost Update** denir. Race condition'ın DB seviyesindeki tezahuru.

Critical Section & Atomicity

Race condition'ın yaşandığı kod parçasına **critical section** (kritik bölge) denir. Çözüm: aynı anda sadece bir thread'in oraya girmesine izin ver. "Tek kişilik tuvalet" gibi — kapiya kilit koyarsın, biri içerideyken diğerleri bekler.

Bir işlem "atomic" ise: arada kesilemez, bütün olarak ya yapılır ya yapılmaz. **count++** atomic değildir (3 işlem). **AtomicInteger.incrementAndGet()** atomic'tir — CPU instruction'ı seviyesinde bölünemez.

Critical section'i korumanın yolları (sıralanmış):

- (1) Immutability — değişmeyen veride race olmaz
- (2) Confinement — veriyi tek thread'e hapset (ThreadLocal)
- (3) Atomic operations — bölünemez işlemler (AtomicInteger, CAS)
- (4) Synchronization — lock ile sıraya sok

Bunları şimdi tek tek işleyeceğiz.

3. Thread Safety

Birden fazla thread aynı anda kullanırsa bile state'i bozulmayan sınıf.

Tanım

Bir sınıf "thread-safe" ise: birden fazla thread aynı anda kullanırsa bile davranışı bozulmaz, state'i tutarlılığını korur. Korumanın **5 ana yolu** var:

Yol 1 — Immutability (degismezlik)

En güvenli yöntem. **Degismeyen veride race condition olmaz.** String, BigDecimal, LocalDateTime, Integer — hepsi immutable. Onları thread'ler arasında gönül rahatlığıyla paylaşabilirsin.

Kendi immutable sınıfını yazma resetesi

```
public final class Para {                                // 1. final class - kalitim engellenir
    private final BigDecimal miktar;                    // 2. tum field'lar final
    private final String parabirim;

    public Para(BigDecimal miktar, String parabirim) {
        this.miktar = miktar;                            // 3. constructor'da set et, baska metod set etmesin
        this.parabirim = parabirim;
    }

    public BigDecimal getMiktar() { return miktar; }      // 4. SETTER YOK!
    public String getParabirim() { return parabirim; }

    public Para ekle(Para other) {
        // 5. Degisiklik gerekirse YENI obje don, mevcudunu degistirme
        return new Para(miktar.add(other.miktar), parabirim);
    }
}
```

Bu sınıf **otomatik thread-safe**. Hiçbir sync gerektirmez. Pure golden rule.

Joshua Bloch: "Classes should be immutable unless there's a very good reason to make them mutable." Effective Java Item 17. Modern Java'da DTO'larını, value object'lerini immutable yaz. Record syntax'i (Java 14+) bunu çok kolaylastırdı.

```
// Java Record - otomatik immutable
public record Para(BigDecimal miktar, String parabirim) {
    public Para ekle(Para other) {
        return new Para(miktar.add(other.miktar), parabirim);
    }
}
```

Yol 2 — Thread Confinement (hapis)

"Bu veri sadece bir thread'e ait, baskısı gormez." Paylaşılmayan veride race olmaz, basit.

Yöntem 1: Local variable. Metod içindeki local değişkenler her thread için ayrı. Bunlara dokunmak güvenli.

Yontem 2: ThreadLocal. Her thread icin ayri kopya tutan ozel container.

```
private static final ThreadLocal<SimpleDateFormat> dateFormat =
    ThreadLocal.withInitial(() -> new SimpleDateFormat("dd/MM/yyyy"));

public String formatla(Date d) {
    return dateFormat.get().format(d);    // her thread kendi format'ini kullanir
}
```

SimpleDateFormat thread-safe degildir — Java standard kutuphanesinin en cok ayagi kayan yerlerinden. ThreadLocal'la sarmalamak klasik cozum (DateTimeFormatter kullanmak daha modern, o thread-safe).

ThreadLocal tehlikesi: Tomcat/Jetty thread pool'u thread'leri yeniden kullanir. Eger ThreadLocal'i **temizlemezszen**, bir request'in verisi sonraki request'e sizar. Memory leak ve **guvenlik aciği!** Her zaman finally'de threadLocal.remove() cagir, ya da Spring'in RequestContextHolder gibi yonetilen mekanizmalari kullan.

Yol 3 — Atomic operations

Java'nin java.util.concurrent.atomic paketi: AtomicInteger, AtomicLong, AtomicReference, vb. Bunlar lock'suz thread-safe operasyonlar sunar.

```
AtomicInteger counter = new AtomicInteger(0);
counter.incrementAndGet();    // atomic, race yok, lock yok
counter.compareAndSet(5, 10); // 5'se 10 yap, atomic
```

Altta **CAS (Compare-And-Swap)** kullanir. CPU'nun bir instruction'i var: "su memory location'da deger X ise Y yap, atomic olarak." Bu donanim destegi sayesinde lock'a gerek kalmaz.

CAS dongusu psevdokod:

```
loop:
    oldValue = read(memory)
    newValue = compute(oldValue)
    if compare-and-swap(memory, oldValue, newValue) succeeds:
        return
    else:
        goto loop    // bu sirada baskasi yazmis, tekrar dene
```

Bu lock-free programlama temelidir. Atomic class'lar yuksek throughput'a sahip, dusuk contention'da harika. Cok yuksek contention'da CAS surekli fail edip retry yapacagi icin synchronized'a kayabilir performans.

Yol 4 — Synchronization (kilitleme)

Klasik cozum: synchronized keyword'u. Bir bloka veya metoda lock koyar, ayni anda sadece bir thread iceri girer.

Synchronized metod

```
class Counter {
    private int count = 0;

    public synchronized void increment() { // metod tamamı lock'lu
        count++;
    }

    public synchronized int get() {
        return count;
    }
}
```

synchronized metod, o objenin (this) **intrinsic lock**'unu (monitor) alır. Static metod ise sınıfın class object'inin lock'unu alır.

Synchronized block

```
public void increment() {
    synchronized (this) { // bu objenin lock'unu al
        count++;
    }
}

// Veya ayrı bir lock objesi:
private final Object lock = new Object();

public void increment() {
    synchronized (lock) { // daha temiz, lock kapsamını netleştirir
        count++;
    }
}
```

Ayrı lock objesi neden tercih edilir? Bircok sebep: (1) lock'in kapsamı acik gozukur, (2) disardan synchronized(theObject) diyerek kimse lock'unu calamaz, (3) farkli mantiklar icin farkli lock'lar kullanabilirsin.

ReentrantLock — synchronized'in akilli kuzeni

java.util.concurrent.locks.Lock interface'i daha esnek. Try-lock, timeout, fairness, interruptable.

```
private final Lock lock = new ReentrantLock();

public void increment() {
    lock.lock();
    try {
        count++;
    } finally {
        lock.unlock(); // FINALLY'de unlock! Yoksa felaket: lock kalir, sistem donar
    }
}

// Timeout ile - bekleme suresi sinirla:
if (lock.tryLock(2, TimeUnit.SECONDS)) {
    try {
        // is yap
    } finally {
        lock.unlock();
    }
} else {
    // lock alinamadi, alternatif yol
}
```

}

synchronized vs ReentrantLock:

synchronized: basit, otomatik unlock (block cikiisinda), JVM optimize ediyor.

ReentrantLock: manuel ama esnek — tryLock, timeout, fairness, interruptible, condition variables.

Default secimin synchronized olsun, gerçekten esnekligin gerekirse ReentrantLock'a gec.

ReadWriteLock — okuma/yazma ayrimi

Birden fazla reader ayni anda okuyabilir, writer single. Read-heavy use case'lerde harika — mesela TSIRPRM cache: 10000 okuma, ara sira guncelleme.

```
ReadWriteLock rwLock = new ReentrantReadWriteLock();
Map<String, String> cache = new HashMap<>();

public String getParam(String key) {
    rwLock.readLock().lock();    // birden fazla reader paralel
    try {
        return cache.get(key);
    } finally {
        rwLock.readLock().unlock();
    }
}

public void updateParam(String key, String value) {
    rwLock.writeLock().lock();    // writer exclusive
    try {
        cache.put(key, value);
    } finally {
        rwLock.writeLock().unlock();
    }
}
```

Yol 5 — Concurrent Collections (hazir cozumler)

Java zaten thread-safe collection'lar veriyor. Kendin yazmana gerek yok.

ConcurrentHashMap

Thread-safe HashMap. Hashtable ve Collections.synchronizedMap()'tan cok daha hizli — cunku tum map'i degil, **bucket bazinda** lock'lar. Yuksek concurrent erisimde harika performans.

```
Map<String, Personel> cache = new ConcurrentHashMap<>();

cache.put("123", personel);           // thread-safe
Personel p = cache.get("123");        // thread-safe

// Atomic compound operations:
cache.computeIfAbsent("456", id -> personelRepo.findById(id)); // thread-safe
```

CopyOnWriteArrayList

Read-heavy listeler icin. Her yazida **tum diziyi kopyalar**. Okuyucular eski diziyi okur, yazici yeni diziyi olusturur, sonra referansi swap eder. Read tarafinda **lock yok** — cok hizli.

Listener listesi, observer'lar gibi seyrek update + sik okuma senaryolarinda kullan. Surekli update varsa felaket olur (her update'te tum dizi kopyalanir).

BlockingQueue

Producer-consumer pattern icin. put() queue doluysa bekler, take() queue bossa bekler. Otomatik flow control.

```
BlockingQueue<Bildirim> kuyruk = new LinkedBlockingQueue<>(1000);

// Producer thread:
kuyruk.put(bildirim); // queue doluysa bekler

// Consumer thread:
while (true) {
    Bildirim b = kuyruk.take(); // queue bossa bekler
    gonder(b);
}
```

ARGUS'ta bildirim sistemine, audit log async yazmaya, batch processing'e cok uygun.

Diger faydali concurrent araclar

ConcurrentLinkedQueue — lock-free queue.

CountDownLatch — "N tane is bitsin, ben de baslayim". Mesela 5 paralel servisten cevap gelene kadar bekle.

CyclicBarrier — "N tane thread bu noktada bulussun, sonra hep birlikte devam etsin".

Semaphore — N adet permit, en fazla N thread iceri girebilir. Rate limiting, connection pool.

Mulakat sorulari — Thread Safety

"HashMap thread-safe mi?"

Hayir. Concurrent yazmada veri kaybolur, daha kotusu Java 7 ve oncesinde infinite loop bile yaratabilirdi (rehash sirasinda). Thread-safe alternatifler: **ConcurrentHashMap** (tercih edilen, segment locking), **Hashtable** (legacy, tum metodlar synchronized, yavas), **Collections.synchronizedMap()** (wrapper, yavas).

"String thread-safe mi?"

Evet, cunku immutable. StringBuffer da thread-safe (sync metod). **StringBuilder thread-safe degil** — single-thread'de daha hizli oldugu icin tercih edilir. Tipik mulakat tuzagi: "hangisi thread-safe?" StringBuilder mi StringBuffer mi — cevap Buffer.

"volatile keyword'u tek basina thread safety saglar mi?"

HAYIR. volatile sadece **visibility** ve **ordering** garantisi verir. Atomicity vermez. volatile int count; count++; hala race condition'a aciktir. count++ atomic degil. Atomicity icin synchronized veya AtomicInteger gerekir.

"Singleton'i thread-safe yapmanin yollari?"

Design Patterns PDF'inde anlatildi: Eager init, Synchronized method, Double-Checked Locking (volatile ile), Bill Pugh holder, Enum singleton. En sik Bill Pugh ve Enum onerilir.

"ThreadLocal niye memory leak yapar?"

ThreadLocal degerleri Thread objesinin icinde tutulur. Thread pool'da thread'ler oldurulmez, geri donulur. remove() cagrilmadan thread havuza geri donerse, deger orada kalir. **Yeni request o thread'i alirsa, eski veriyi gorebilir** (guvenlik aciği) veya memory leak olur. Cozum: **finally'de daima remove() cagir.**

"Stateless servis neden thread-safe?"

State tutmuyorsa paylasacak veri yok, race condition imkansiz. Spring'in @Service bean'leri singleton'dir ama stateless oldugu icin guvenli. State tutmaya basladiginda (instance field'a yazmaya basladiginda) thread safety problemi cikar.

"Bir metodu synchronized yapinca tum sinif lock'lanir mi?"

Hayir. Instance metod synchronized ise **o object'in monitor'u** lock'lanir — ayni siniftan baska object'lerin metodlari paralel calismaya devam eder. Static metod synchronized ise **Class object'i** lock'lanir — tum sinif duzeyinde lock olur.

4. Deadlock, Livelock, Starvation

Lock'lar ise yarar... ta ki kendi kuyruguna takilana kadar.

Deadlock — kilitlenme

İki (veya daha fazla) thread birbirinin tuttuğu lock'u bekliyor, ikisi de diğerinin lock'u bırakmasını istiyor. Ebediyen beklerler. Sistem yarıda durur, kullanıcı "sistem dondu" diye yazılım ekibini arar.

Coffman conditions — deadlock'in 4 şartı

Bir deadlock'in olması için DORDU birden gerekiyor:

- (1) **Mutual Exclusion:** kaynak paylaşılamaz — bir lock'u aynı anda 2 thread alamaz.
- (2) **Hold and Wait:** thread bir lock tutarken başka bir lock istiyor.
- (3) **No Preemption:** bir lock zorla alınamaz — sahibi gönüllü bırakmalı.
- (4) **Circular Wait:** lock isteme zinciri daireseldir (T1 -> T2 -> T1).

Bunlardan **birini** kirarsan deadlock olamaz. Pratikte en kolay kırılana (4) Circular Wait.

Klasik kod örneği

```
Thread 1:
    synchronized(A) {
        synchronized(B) {      // B'yi istiyor
            // is yap
        }
    }

Thread 2:
    synchronized(B) {
        synchronized(A) {      // A'yi istiyor
            // is yap
        }
    }
```

Senaryo:

```
t=0:  T1 A'yi tutar, B'yi ister.
t=0:  T2 B'yi tutar, A'yi ister.
t=1:  T1 B'yi bekliyor.    T2 A'yi bekliyor.
...   sonsuza kadar...
```

ARGUS senaryosu — hesap transferi

İki kullanıcı aynı anda transfer yapıyor. T1: hesap 100'den hesap 200'e. T2: hesap 200'den hesap 100'e.

```
void transfer(Hesap from, Hesap to, BigDecimal miktar) {
    synchronized (from) {
        synchronized (to) {
            from.azalt(miktar);
            to.arttir(miktar);
        }
    }
}
```

```
// T1: transfer(hesap100, hesap200, ...) -> 100'u lock'lar, 200'u bekler
// T2: transfer(hesap200, hesap100, ...) -> 200'u lock'lar, 100'u bekler
// DEADLOCK
```

Cozum 1 — Lock Ordering (kilit sirasi)

Her zaman **ayni sirayla** lock al. Mesela hesap ID'sine gore: kucuk ID once, buyuk ID sonra. Boyle yaparsan circular wait imkansiz, deadlock yok.

```
void transfer(Hesap from, Hesap to, BigDecimal miktar) {
    Hesap first = from.getId() < to.getId() ? from : to;
    Hesap second = from.getId() < to.getId() ? to : from;

    synchronized (first) {
        synchronized (second) {
            from.azalt(miktar);
            to.arttir(miktar);
        }
    }
}
```

Her transfer ayni sirayi izler — kucuk ID once lock'lanir. Iki transfer ayni iki hesabi farkli sirada lock'lamaya calismaz, deadlock imkansiz.

Cozum 2 — tryLock with timeout

ReentrantLock.tryLock() ile timeout koy. Belirli sure lock alinamazsa vazgeç, retry et.

```
if (lockA.tryLock(1, TimeUnit.SECONDS)) {
    try {
        if (lockB.tryLock(1, TimeUnit.SECONDS)) {
            try {
                // is yap
            } finally { lockB.unlock(); }
        } else {
            // B alinamadi, vazgeç - retry et
        }
    } finally { lockA.unlock(); }
}
```

Bu yaklasim deadlock'i kirar (No Preemption sartini bypass eder) ama livelock riskine acar — surekli vazgeçip tekrar denerlerse hicbir ilerleme olmaz.

Cozum 3 — Tek lock, daha az granularite

Iki lock yerine bir tane buyuk lock kullan. Performans dususebilir ama deadlock yok. Trade-off.

Livelock — naziklesik kilit

Thread'ler aslinda calisiyor (deadlock gibi donmus degil) ama **hicbir ilerleme yapmıyorlar** — birbirlerine yol vermek icin surekli geri cekiliyorlar.

Klasik benzetme: iki nazikli adam dar kapinin onunde. "Buyur sen gec" — "Yok yok sen gec" — "Israr ediyorum, sen gec" — "Ben de ısrar ediyorum..." — saatlerce hicbiri gecmez.

Kod uzerinde: iki thread surekli tryLock yapip basaramayınca vazgeçiyor, ayni anda yeniden deneme yapip yine fail oluyor — sonsuz dans. Cozum: **random backoff** —

vazgecince rastgele bir süre bekle, sonra dene. Network protokollerinde (Ethernet, TCP) bu standart çözümdür.

Starvation — açlık

Bir thread sürekli düşük öncelikli kalıyor, CPU veya lock'u hiç alamıyor. Sistem çalışıyor ama o thread sonsuza dek bekliyor.

Sebepler: thread priority dengesi, fairness eksikliği, **writer starvation** (read-heavy ReadWriteLock'ta yazıcı hep beklemede kalır çünkü reader'lar bitmiyor).

Çözüm: **fair lock** kullan. `new ReentrantLock(true)` ile FIFO queue oluşur, her thread sırayla alır. Performans biraz düşer ama starvation yok.

5. Database Locking

Java'da nasıl thread'leri koruyorsak, DB'de de transaction'ları böyle koruyoruz.

İki yaklaşım — Pessimistic vs Optimistic

Es zamanlı yazmaları yönetmenin iki felsefesi var:

Pessimistic Locking — "catisma olur, baştan kilitleyelim"

Veriyi okurken hemen lock al. Baska transaction o veriye dokunamaz (lock tipi ne kadar sıkı seçildiğine bağlı). Lock transaction sonuna kadar tutulur.

```
// SQL Server / PostgreSQL:
SELECT * FROM tblper WHERE id = 42 FOR UPDATE;
-- bu satır lock'landı, baska transaction modify edemez

-- JPA / Hibernate:
@Lock(LockModeType.PESSIMISTIC_WRITE)
Personel findById(Long id);
```

Avantaj: Lost update olmaz, garanti. **Dezavantaj:** Diğer transaction'lar bekler, throughput düşer, deadlock riski artar.

Ne zaman: Catisma sık, kayıp pahalı. Finansal transferler, stok yönetimi, biletleme.

Optimistic Locking — "muhtemelen catisma olmaz, sonunda kontrol ederiz"

Veriyi okurken lock alma. Update sırasında **version** sütununu kontrol et. Eğer arada baskası güncellemisse version uyusmaz, exception fırlar. Retry mantığı devreye girer.

```
@Entity
public class Izin {
    @Id
    private Long id;

    @Version
    private Long version;    // Hibernate her update'te otomatik artırır

    private String durum;
    // ...
}
```

Senaryo:

```
t=0: T1 izin'i yukler -> version=1
t=0: T2 aynı izin'i yukler -> version=1
t=1: T1 update yapar, commit -> version=2 olur
t=2: T2 update yapmaya calisir:
      UPDATE izin SET ... WHERE id=42 AND version=1
      -> 0 row affected (cunku version=2 şimdi)
      -> Hibernate OptimisticLockException firlatir
```

Avantaj: Lock yok, throughput yüksek. **Dezavantaj:** Catisma olunca retry sorumluluğu sende. Kullanıcı yine aynı işi yapsın diye uyarı verebilirsin.

Ne zaman: Catisma seyrek, retry ucuz. Form save'leri, profil update'leri, kullanıcı tercihleri.

ARGUS senaryosu: 2 İK uzmanı aynı izin kaydını editliyor. Optimistic uygun — biri kaydettince diğere "Bu kayıt son okumandan beri değişti, lütfen yenile ve tekrar dene" diyebilirsiniz. Veri kaybı olmaz, kullanıcı ne olduğunu bilir. Pessimistic koysaydık ikincisi 30 saniye lock'a takılırdı — kullanıcı bilemezdi neden donmuş.

DB lock türleri (teknik detay)

DB'ler farklı granularite ve modlarda lock kullanır. Mulakatta detayı sorulabilir:

Granularite:

Row lock — sadece o satır kilitlenir (en yaygın)

Page lock — DB sayfa düzeyinde (SQL Server)

Table lock — tüm tablo (DDL operasyonlarında)

Mode:

Shared (S) lock — başka transaction'lar da read için S lock alabilir, X lock alamaz

Exclusive (X) lock — kimse başka lock alamaz

Update (U) lock — gelecekteki X lock'a yol açan kısmi koruma

Mulakat soruları — DB Locking

"Pessimistic mi optimistic mi tercih edersin?"

Soruyu "catisma frekansı neye benziyor?" diye geri çevirmen iyi. Catisma seyrekse optimistic — performans yüksek, kullanıcıya net feedback verir. Catisma sık veya kayıp pahalı ise pessimistic — garanti olsun. ARGUS'taki cogsu CRUD optimistic'e uygun, finansal/stok benzeri yerlerde pessimistic.

"SELECT FOR UPDATE'in NOWAIT versiyonu nedir?"

SELECT ... FOR UPDATE NOWAIT — eğer satır baskası tarafından kilitliyse, beklemez, hemen exception fırlatır. Kullanıcı 30 saniye donmasın diye kullanılır; "baskası şimdi bunu düzenliyor" diye anlık feedback verebilirsin.

"@Version atomik mi çalışır?"

Evet. Hibernate UPDATE ... WHERE id=? AND version=? şeklinde çalışır. WHERE koşulu DB tarafından atomik kontrol edilir. Version eşleşmiyorsa 0 row update edilir, Hibernate bunu OptimisticLockException olarak yorumlar.

"Optimistic locking'de exception aldım, ne yaparım?"

İki seçenek: (1) **Retry et** — veriyi tekrar yükle, kullanıcının değişikliğini üzerine uygula, kaydet. Otomatik olabilir (saniyelerle sınırlı, infinite değil). (2) **Kullanıcıya yansıt** — "Bu kayıt X tarafından güncellendi, yenilemek ister misin?" Hangisini seçersin? **Kullanıcı editliyorsa** ikincisi (kullanıcı kararı kaybetmek istemez), **otomatik işlemse** birincisi.

"DB deadlock'i nasıl tespit edersin ve cozersin?"

Modern DB'ler (SQL Server, PostgreSQL, MySQL InnoDB) deadlock'i otomatik tespit eder ve transaction'lardan birini **kurban** seçip rollback yapar. JDBC tarafında bunu SQLException olarak alırsın (error code'a bak), retry mantığı yazabilirsin. Önleme: aynı lock sırası, kısa transaction'lar, row-level lock kullan, table-level'dan kaç.

6. Java Memory Model & Atomics

CPU cache ile main memory arasında sevgisiz bir ilişki var. JMM bu ilişkinin kuralları.

Once sezgi — neden bir "memory model" var?

Modern CPU'larda her çekirdek kendi cache'ini kullanır (L1, L2, L3). Bir thread bir field'a yazınca, o yazım **öncelikle CPU cache'ine gider**, sonra eventually main memory'ye flush olur. Başka bir çekirdekteki thread ise **kendi cache'inden** okur — eski değeri görebilir.

Ayrıca JIT compiler ve CPU performans için **instruction reordering** yapar. Yazdığın kodun sırası gerçekte aynı sırayla çalışmaz.

Bu iki şey single-thread programlarda görünmez (programcı için transparent), **multi-thread'de cinnetlik** yaratır. Java Memory Model (JMM) bu kaosa kural koyar.

volatile — visibility ve ordering garantisi

volatile keyword'u iki garanti verir:

(1) **Visibility**: bir thread'in yazdığı anında diğer thread'ler tarafından görülür. CPU cache bypass edilir.

(2) **Ordering**: volatile field'a yazım/okuma etrafında instruction reordering yapılmaz (memory barrier).

Klasik visibility problemi:

```
class Flag {
    private boolean running = true;

    public void stop() {
        running = false;
    }

    public void doWork() {
        while (running) {
            // iş yap
        }
        // JIT compiler running'i CPU register'ine optimize edebilir
        // Başka thread stop() çağırırsa burada GÖRÜLMİYEBİLİR
        // Sonsuz döngü! Bug bulunması çok zor.
    }
}
```

Çözüm: `private volatile boolean running = true;` — visibility garantisi.

volatile atomicity SAGLAMAZ. Sadece visibility ve ordering. `volatile int count; count++;` hala race condition'a açıktır. `count++` üç adımlı, araya başka thread girebilir.

happens-before — JMM'in temel kavrami

JMM "A operasyonu B'den happens-before ise, A'nin etkisi B tarafından gorulur" der. Eger iki operasyon arasinda happens-before iliskisi yoksa, sirayi JVM/CPU istedigi gibi belirleyebilir.

Happens-before garantisi veren durumlar:

Program order: ayni thread icindeki ifadeler yazildiklari sirayla happens-before.

Monitor lock: bir lock'in unlock'i, ayni lock'in sonraki lock'undan happens-before.

Volatile: bir volatile field'a write, sonraki read'inden happens-before.

Thread.start(): start() cagrisi, yeni thread'in herhangi bir operasyonundan happens-before.

Thread.join(): bitcek thread'in son operasyonu, join()'in donmesinden happens-before.

Final field: constructor'in bitisi, final field'in herhangi bir read'inden happens-before.

Mulakat altin cumlesi: "volatile gerekli, cunku CPU cache ve instruction reordering yuzunden bir thread'in yazdigi sey diger thread tarafından gorulmeyebilir. JMM'nin happens-before kurali ile volatile bu garantiyi verir."

Atomic classes — lock-free thread safety

java.util.concurrent.atomic paketi atomic operasyonlar sunar. Lock kullanmadan thread-safe, yuksek performans.

AtomicInteger / AtomicLong / AtomicBoolean

```
AtomicInteger counter = new AtomicInteger(0);

counter.incrementAndGet(); // i++ gibi ama atomic, race yok
counter.decrementAndGet(); // i--
counter.addAndGet(5);      // i += 5
counter.compareAndSet(5, 10); // i == 5 ise i = 10 yap, atomic
```

AtomicReference — herhangi bir nesne icin atomic referans

```
AtomicReference<Personel> guncelPersonel = new AtomicReference<>();

// Atomic update:
guncelPersonel.set(yeniP);

// Compare-and-set:
guncelPersonel.compareAndSet(eskiP, yeniP);

// updateAndGet – atomic transform:
guncelPersonel.updateAndGet(p -> p.withAd("Yeni Ad"));
```

CAS (Compare-And-Swap) — alt yapi

Atomic class'larin ozellestirici ozelligi: lock kullanmadan thread-safe. Nasil? **CPU'nun yerli compare-and-swap instruction'i** ile. Modern x86 CPU'larda CMPXCHG, ARM'da LDREX/STREX. JVM bunlari kullanir.

```
// CAS dongusu pseudo-kodu:
```

```
do {
    int eski = atomic.get();
    int yeni = hesapla(eski);
} while (!atomic.compareAndSet(eski, yeni));

// Eger arada baskasi degistirdiyse, CAS basarisiz, retry.
```

Bu lock-free programlama'nin temelidir. Dusuk-orta contention'da synchronized'dan cok daha hizli. Cok yuksek contention'da CAS retry'lari fazlalasir, performans dusebilir — o noktada lock'a donmek mantikli.

LongAdder — yuksek contention icin alternatif

Cok thread surekli increment yapiyorsa, AtomicLong'da CAS retry'lar artar. LongAdder her thread icin ayri cell tutar, sum() cagirilince toplar. Cok daha hizli, ama anlik tutarli okuma vermez.

Mulakat sorulari — JMM & Atomics

"volatile ile synchronized arasinda fark?"

volatile: visibility + ordering, atomic operasyon icin yeterli degil. Read/write basit ise (boolean flag, reference) yeterli. **synchronized:** atomicity + visibility + ordering, daha pahali ama kapsamli. compound operasyonlar (count++) icin gerekli.

"AtomicInteger vs synchronized counter — hangisi hizli?"

Dusuk-orta contention'da **AtomicInteger** cok daha hizli (lock-free, CAS). Cok yuksek contention'da (yuzlerce thread surekli increment) CAS retry'lar fazlalasir, synchronized'a yaklasir veya geri kalir. Asiri yuksek contention icin LongAdder.

"CAS'in ABA problemi nedir?"

Bir thread X degerini okur. Diger thread X'i Y yapar, sonra tekrar X yapar. Ilk thread CAS(X, Z) yapar, **basarili olur** — cunku deger gercekten X. Ama arada degisme oldu! Bu ABA. AtomicReference'ta cok onemli olabilir (linked list operasyonlarinda). Cozum: AtomicStampedReference — deger + version numarasi tutar.

"final field'lerin thread-safety'siyle ne ilgisi var?"

final field'lar JMM tarafından ozel olarak korunur: constructor bittikten **sonra** hicbir thread "yarim olusmus" obje gormez. Bu yuzden immutable nesneler thread-safe — final field'lar tum thread'lerce dogru gorulur, kalitiplari publication problemi olmaz.

"Double-checked locking neden volatile gerektirir?"

Design Patterns PDF'inde Singleton bolumunde anlattik. Ozet: instance = new K() atomic degil (uc adim: alloc, set ref, run constructor). JIT 2 ile 3'u reorder edebilir. Baska thread instance != null gorur ama yari olusmus objeye dokunur. volatile happens-before garantisi vererek reordering'i engeller.

Mulakat Cheat Sheet

Tek cumlede her kavram

Transaction: bir DB islem paketi — ya hepsi olur ya hicbiri.

ACID: Atomicity (hep/hic), Consistency (kural), Isolation (yalitim), Durability (kalici).

Isolation Level: es zamanli transaction'larin birbirini ne kadar gordugunu belirler.

Race Condition: iki thread ayni veriye dokunup birinin sonucu kayboldugunda.

Thread Safety: sinifin coklu thread'lerce dogru calismasi.

Lock: tek anda bir thread'in critical section'a girmesini saglayan kilit.

Deadlock: iki thread birbirinin lock'unu bekleyip donmesi.

volatile: visibility ve ordering garantisi (atomicity DEGIL).

CAS: Compare-And-Swap — donanim destekli atomic operasyon, lock-free temeli.

Phenomenas ve isolation level'lar (ezberle!)

Dirty Read → READ UNCOMMITTED'de olur, READ COMMITTED'de cozulur

Non-Repeatable Read → READ COMMITTED'de olur, REPEATABLE READ'de cozulur

Phantom Read → REPEATABLE READ'de olur, SERIALIZABLE'da cozulur

"Acim vardi, cozumumu uyguladim" cumlelerin

Transaction: "Izin onayinda 5 farkli tablo guncellemesi vardi, biri fail edince digerlerinin geri alinmaması tutarsizlik yaratıyordu, @Transactional ile tek paket yaptik."

Race Condition: "İki HR uzmanı aynı personelin izin gününü update edebiliyordu, lost update oluyordu. @Version ile optimistic locking ekledik, ikinci kullanıcı uyarı alıyor ve yeniliyor."

Thread Safety: "SimpleDateFormat'i singleton instance olarak kullanıyorduk, sporadik patlıyordu. Thread-safe değildi. ThreadLocal'la sarmaladık, DateTimeFormatter'a gectik."

Deadlock: "Transfer işlemlerinde iki hesap lock'lanırken sırasız alınıyordu, deadlock yasadık. Hesap ID'sine göre lock ordering uyguladık, deadlock kayboldu."

Mulakatçı tuzakları — sık gelen sorular

ACID'i hangisi en zor → Isolation (level seçimi senin sorumluluk)

@Transactional public olmazsa → sessizce çalışmaz

@Transactional self-invocation → proxy bypass, çalışmaz

@Transactional checked exception → default rollback YAPMAZ

HashMap thread-safe mi → HAYIR. ConcurrentHashMap kullan

StringBuilder vs StringBuffer → Buffer thread-safe, Builder degil

volatile atomicity verir mi → HAYIR, sadece visibility/ordering

count++ atomic mi → HAYIR, 3 islem (read-modify-write)

Singleton thread safety → Bill Pugh holder veya Enum oneril

Optimistic vs Pessimistic → catisma frekansina gore

Deadlock'in 4 sarti → mutex, hold-wait, no-preempt, circular wait

Deadlock cozumu → lock ordering (circular wait'i kir)

Son taktik: bu konularda mulakatci hep "kendi kodunda nerede karsilastin?" diye sorar. Hayali ornek verme — gercek yasanmis bir senaryoyu anlat. Yukarida verdigim cumleler ARGUS'ta gercekten denk gelebilecek senaryolar. Birini kendi diline cevrip yas — mulakatci bunu cok degerli bulur, ezberden gelen cevap degil deneyimden gelen cevap olarak algilar.

Bol sans Batur. Konular bol ama yukaridakileri yutmussan, sana sorulacak %90'i ele aldin demek.